

W13D01 — PostgreSQL Setup & Connection

ML Engineer Journey · Phase 4: Database & Observability · 1 Jun 2026

1. Kenapa Aplikasi Production Butuh Database

Bayangkan sebuah dapur. Setiap kali kompor dimatikan, semua bahan di meja lenyap — tidak tersisa apapun. Itulah yang terjadi pada aplikasi yang hanya menyimpan data di memory atau file di dalam container. Setiap restart container: semua prediksi, semua log request, semua hasil komputasi — hilang selamanya.

PostgreSQL adalah **lemari es** di dapur itu. Data tersimpan di disk, persisten, tidak peduli berapa kali container di-restart atau di-replace.

Aspek	Memory / File di Container	PostgreSQL (disk)
Saat container restart	Data hilang selamanya	Data tetap ada
Saat container dihapus	Data hilang selamanya	Data tetap ada (di volume)
Cocok untuk production	Tidak	Ya — standar industri
Sifat penyimpanan	Volatile (RAM/filesystem sementara)	Persistent (disk, WAL log)

2. PostgreSQL — Konsep Fundamental

PostgreSQL adalah **Relational Database Management System (RDBMS)** open-source yang menyimpan data dalam bentuk tabel, baris, dan kolom. Data tersimpan di disk menggunakan mekanisme **WAL (Write-Ahead Log)** — setiap perubahan dicatat di log dulu sebelum diterapkan ke data file. Ini yang menjamin data tidak korup meski terjadi crash mendadak.

Hubungan dengan materi sebelumnya: di W10D03 kamu belajar named volume untuk data persistence. Di W10D04 kamu belajar environment variables untuk secrets. Hari ini keduanya dipakai sekaligus — named volume untuk menyimpan data PostgreSQL, env vars untuk credentials.

3. Menjalankan PostgreSQL via Docker

Perintah run (PowerShell — backtick sebagai line continuation):

```
docker run -d `
```

```
--name pg-server `
```

```
-e POSTGRES_PASSWORD=superrahasia `
```

```
-p 5432:5432 `
```

```
-v pgdata:/var/lib/postgresql `
```

```
postgres:latest
```

Penjelasan setiap flag:

Flag	Nilai	Fungsi
-d	—	Detached mode — container jalan di background
--name	pg-server	Nama container agar mudah direferensikan
-e POSTGRES_PASSWORD	superrahasia	Password superuser postgres
-p	5432:5432	Forward port host:container
-v	pgdata:/var/lib/postgresql	Named volume — data persisten di host
postgres:latest	—	Image PostgreSQL (production: pin versi, misal postgres:15)

Catatan kritis PostgreSQL 18+: Mount point berubah dari `/var/lib/postgresql/data` menjadi `/var/lib/postgresql` (satu level di atas). Salah mount point = container crash dengan error format tidak kompatibel.

Verifikasi container berjalan:

```
docker ps
```

```
docker logs pg-server
```

```
# Baris terakhir yang diharapkan:
```

```
# LOG: database system is ready to accept connections
```

4. Named Volume & Bukti Persistensi

Named volume adalah area penyimpanan yang **dikelola Docker di host machine**, terpisah sepenuhnya dari filesystem container. Saat container dihapus dengan `docker rm`, volume TIDAK ikut terhapus — kecuali kamu eksplisit menjalankan `docker volume rm nama_volume`.

Ini yang terjadi saat eksperimen persistensi hari ini:

```
-- DALAM CONTAINER PERTAMA --
```

```
CREATE TABLE test_persistence (
```

```
  id SERIAL PRIMARY KEY,
```

```
  message TEXT NOT NULL,
```

```
  created_at TIMESTAMP DEFAULT NOW()
```

```
);
```

```
INSERT INTO test_persistence (message) VALUES ('data ini harus tetap ada setelah restart');
```

```
-- HAPUS CONTAINER --
```

```
docker stop pg-server && docker rm pg-server
```

```
-- JALANKAN CONTAINER BARU DENGAN VOLUME YANG SAMA --
```

```
docker run -d --name pg-server -v pgdata:/var/lib/postgresql postgres:latest
```

```
-- DALAM CONTAINER BARU -- DATA MASIH ADA --
```

```
SELECT * FROM test_persistence;
```

```
-- id=1, message='data ini harus tetap ada setelah restart' ✓
```

Kesimpulan: yang menyimpan data adalah **volume**, bukan container. Container hanyalah proses yang me-mount volume tersebut.

5. Perintah psql yang Wajib Diketahui

Perintah	Fungsi	Catatan
\l	List semua database	Meta-command, tidak perlu titik koma
\c nama_db	Connect ke database lain	Meta-command
\dt	List semua tabel di database aktif	Meta-command
\d nama_tabel	Struktur tabel (kolom, tipe, constraint)	Meta-command
\conninfo	Info koneksi saat ini	User, host, port, database
\q	Keluar dari psql	Meta-command
SELECT current_user;	User yang sedang login	SQL — butuh titik koma
SELECT version();	Versi PostgreSQL	SQL — butuh titik koma
SELECT * FROM pg_database;	List database via SQL	Bisa dipakai dari kode Python

Perbedaan mendasar: perintah \ (backslash) adalah **meta-command psql** — hanya bisa dipakai di sesi psql interaktif, tidak perlu titik koma. Perintah SQL standar (SELECT, CREATE, INSERT) harus diakhiri titik koma dan bisa dijalankan dari kode Python.

6. Koneksi Python ke PostgreSQL via psycopg2

psycopg2 adalah library Python paling populer untuk koneksi ke PostgreSQL. Pola koneksi yang benar selalu menggunakan **try/except/finally** untuk memastikan koneksi selalu ditutup — bahkan saat terjadi error.

```
import psycopg2
```

```

from psycopg2 import OperationalError


DB_CONFIG = {
    "host": "localhost",
    "port": 5432,
    "database": "postgres",
    "user": "postgres",
    "password": "superrahasia",
}


def get_connection():
    # psycopg2.connect() membuka TCP connection ke PostgreSQL
    return psycopg2.connect(**DB_CONFIG)


def run_query(conn):
    with conn.cursor() as cur:
        # cursor = posisi eksekusi query
        cur.execute("SELECT current_user, current_database();")
        user, db = cur.fetchone()  # fetchone = ambil satu baris

        cur.execute("SELECT table_name FROM information_schema.tables "
                    "WHERE table_schema = 'public';")
        tables = cur.fetchall()  # fetchall = ambil semua baris


def main():
    try:
        conn = get_connection()
        run_query(conn)
    except OperationalError as e:
        # OperationalError: host tidak terjangkau, password salah, db tidak ada
        print(f"Connection failed: {e}")
    finally:
        # finally SELALU dieksekusi – koneksi SELALU ditutup

```

```
if 'conn' in locals() and conn:
```

```
    conn.close() # Wajib! Default max_connections PostgreSQL = 100
```

Kenapa finally itu wajib: PostgreSQL default punya batas 100 koneksi. Kalau koneksi tidak ditutup, setiap request membuka koneksi baru. Dalam hitungan menit di production API, kamu akan dapat error FATAL : remaining connection slots are reserved. Di production, gunakan connection pooling (PgBouncer atau SQLAlchemy pool) untuk mengelola koneksi secara efisien.

7. Praktik Baik: Pin Versi Docker Image

Image Tag	Cocok untuk	Risiko
postgres:latest	Eksperimen cepat saja	Breaking change tanpa peringatan
postgres:18	Development (major pinned)	Minor/patch updates otomatis
postgres:15-alpine	Production (LTS + lightweight)	Minimal — disarankan
postgres:15.7	Production (full pin)	Tidak ada — paling aman

Dalam docker-compose.yml production, selalu pin ke versi spesifik:

```
services:
```

```
  db:
```

```
    image: postgres:15-alpine # ← pin major+variant, bukan latest
```

```
    environment:
```

```
      POSTGRES_USER: mluser
```

```
      POSTGRES_PASSWORD: ${DB_PASSWORD} # ← dari .env, jangan hardcode
```

```
      POSTGRES_DB: mladb
```

```
    volumes:
```

```
      - pgdata:/var/lib/postgresql/data
```

```
    ports:
```

```
      - "5432:5432"
```

```
volumes:
```

```
  pgdata:
```

8. Ringkasan Konsep Kunci Hari Ini

Named volume vs container filesystem

Data di dalam container hilang saat container dihapus. Named volume disimpan di host, dikelola Docker, tidak ikut terhapus bersama container.

Mount point PostgreSQL 18+

PG 18+ menggunakan `/var/lib/postgresql` (bukan `/var/lib/postgresql/data`). Salah mount point = container crash.

psql meta-commands vs SQL

Backslash commands (`\l`, `\dt`, `\q`) hanya untuk sesi interaktif psql. SQL standar (`SELECT`, `CREATE`) bisa dijalankan dari kode Python.

Pola koneksi try/finally

Selalu tutup koneksi di blok finally. PostgreSQL default max 100 koneksi — resource leak = production outage.

Jangan pakai latest di production

Pin versi image Docker untuk konsistensi antar environment dan menghindari breaking change otomatis.